

ECE3623 Embedded System Design Laboratory

ADC LAB

In this Lab the PmodAD1 analog-to-digital converter (ADC) SPI peripheral from the Digilent Vivado IP Library is to be configured and used to perform measurements on an analog test signal.

The Zynq SPI Peripherals ADC and DAC Slides posted on Canvas is the reference for the PmodAD1 ADC. The necessary Lab 5 files are also posted on Canvas. The PmodAD1 Vivado SDK example project AD1Pmod.c is shown below and is also presented in the lecture.

In this Lab you are to use the Digilent Discovery 2 Board and the Waveforms application to generate an analog test signal interfaced to channel 1 of the PmodAD1 ADC. You should have installed and verified the performance of the Waveforms software. The Tasks for this Laboratory are as follows:

1. Execute the example PmodAD1 project in Vivado SDK by following the SPI PowerPoint. The PmodAD1 is connected to JE on the Zybo Board. Setup the Discovery 2 and input a DC voltage (0V, 1V, 2V, 3V and 4V) from the Digilent Discovery 2 Waveform Generator to channel 1 of the ADC. What is the exact saturation voltage for the ADC?
2. Modify the PmodAD1 project.

a) In the Vivado hardware block diagram, add the LEDs as a GPIO, and then in SDK, map the 12-bit ADC output to the 4-bit LEDs. For example, if the ADC output is between 0-???, then the LED pattern should be 0000. Conversely, if the ADC output is between ???-4096, then the LED pattern should be 1111. Determine and map all the values in between the 12-bit range 0-4096, so that the LEDs will illuminate based on the ADC values. (Hint: divide the range of the range of the 12-bit ADC by the number of possible outcomes of the 4-bit LEDs)

b) Input a unipolar ramp waveform from the Digilent Discovery 2 Waveform Generator to channel 1 of the ADC. The ramp waveform should periodically rise from 0v – saturation voltage determined in part 1 above. Make the period of the ramp waveform slow enough to be sampled by the ADC at least 16 times during each period. This will force the LEDs to count up with each ADC sample.

```
//AD1Pmod.c
```

**(NOTE: THIS CODE CAN BE USED TO COMPLETE STEP 1,
BUT MUST BE MODIFIED TO ADD GPIO TO THE LEDS TO
COMPLETE STEP 2)**

```
#include "xparameters.h"
```

```
#include "xil_io.h"
```

```
#include
```

```
#include "platform.h"
```

```
#include "xil_printf.h"
```

```
#include
```

```
//AD1Pmod from Address Editor in Vivado
```

```
#define AD1acq 0x43C00000 //AD1 acquisition
```

```
#define AD1dav 0x43C00004 //AD1 data available
```

```
#define AD1dat1 0x43C00008 //AD1 channel 1 data
```

```
#define AD1dat2 0x43C0000C //AD1 channel 2 data
```

```
int main(void)
```

```
{
```

```
    int adcdav; //ADC data available
```

```
    int adcdat1; //ADC channel 1 data
```

```
    int adcdat2; //ADC channel 2 data
```

```
    xil_printf("\n\rStarting AD1 Pmod demo test...\n");
```

```
    Xil_Out32(AD1acq,0);
```

```
    sleep(5);
```

```
    while (1)
```

```
    {
```

```
        Xil_Out32(AD1acq,0); //ADC stop acquire
```

```
        adcdav=Xil_In32(AD1dav); //ADC available?
```

```
        while(adcdav==1)
```

```
            adcdav=Xil_In32(AD1dav);
```

```
        while(1)
```

```
        {
```

```
            Xil_Out32(AD1acq,1); //ADC acquire
```

```
            while(adcdav==0)
```

```
                adcdav=Xil_In32(AD1dav); //ADC data?
```

```

Xil_Out32(AD1acq,0); //ADC stop acquire

adcdat1=Xil_In32(AD1dat1);

adcdat2=Xil_In32(AD1dat2);

while(adcdav==1)

adcdav=Xil_In32(AD1dav); //ADC reset?

xil_printf("%d Ch1 %d, Ch2 \n", adcdat1, adcdat2);

sleep(1);

//12bit to 4bit Mapping Code Goes Here

If(adcdat1<255) led=0b0000;

Else If(adcdat1<511) led=0b0001;

.

.

.

Else led=0b1111;

Xgpio_discretewrite(&LEDinst;, 1, led);

}

}

}

```

FALL 2023